This line adds the same test helpers. (Minitest fans should note that from here out the code directory for this book will often have Minitest versions of tests in the text, even though I won't specifically call out all of them.)

Adding Users and Roles

Now that you have Devise installed, let's see how you can use testing to expose security issues.

The most basic security issue is user login. Since your application involves projects that would presumably be limited to a specific, private set of users, it makes sense that you would need to be logged in to access the application. This is testable logic—a logged-in user can access a page, whereas any random person who happens across the page cannot.

Here's an integration test for the project index page:

security/01/spec/system/user_and_role_spec.rb

```
Line 1  require "rails_helper"
-
-  RSpec.describe "with users and roles" do
-
5    def log_in_as(user)
-      visit new_user_session_path
-      fill_in("user_email", with: user.email)
-      fill_in("user_password", with: user.password)
-      click_button("Log in")
10   end
-
-    let(:user) { User.create(email: "test@example.com", password: "password") }
-
-    it "allows a logged-in user to view the project index page" do
15     log_in_as(user)
-      visit(projects_path)
-      expect(current_path).to eq(projects_path)
-    end
-
20   it "does not allow a user to see the project page if not logged in" do
-      visit(projects_path)
-      expect(current_path).to eq(user_session_path)
-    end
-
25 end
```

This test uses Capybara, and you've seen most of the component parts before. However, this test has the first of several answers to the question, "How do I simulate a user login in an automated test?" In the helper method `log_in_as` on line 5, you simulate a user login by actually going through the steps of a user login. The method uses Capybara and the standard Devise login route and